

Refactoring

viernes, 3 de abril de 2026 12:26

Es la acción de **mejorar el código** sin cambiar lo que hace.
Se aplica para **arreglar bad smells**

- Mejora
- legibilidad
 - mantenimiento
 - Diseño

Es correcto modificar los test despues de hacer refactoring?
No, generalmente no es correcto modificar los test porque el refactoring no debe cambiar el comportamiento, solo modifica si estaban mal diseñador o dependían de detalles internos

Extract method:

se muestra como tomar un bloque de código y convertirlo en un método separado para mejorar legibilidad

Inline Method:

explica cuando conviene eliminar un método y reemplazarlo directamente por su contenido

Move Method:

detallado con ejemplos de cuando un método vive en la clase equivocada, mueve el metodo a la clase correcta

Move Field:

como trasladar atributos a la clase que realmente los usa

Rename Method:

importancia de nombre claros y como renombrar para mejor comprensión

Pull Up Method/Field:

casos donde mover la lógica común a la super clase

Push Down Method:

traslada el comportamiento especifico a las subclases

Extract Class:

dividir una clase muy grande en varias mas pequeñas

Inline Class:

como eliminar una clase que no aporta valor

Replace Conditional with

Polymorphism :

elimina condicionales largos y los reemplaza por herencia y métodos específicos en subclases

Introduce Parameter Object:

un método recibe demasiados parámetros se agrupan en un objeto

Replace Magic Number with Symbolic

Constant:

reemplaza números en el código por constante con nombre

Self Encapsule Field :

obliga a acceder a atributos mediante getters y setters incluso en la misma clase

Form Template Method:

extrae un método plantilla que define la estructura de un algoritmo dejando que las subclases implementen los pasos específicos

Introduce Null Object:

se crea una clase especial que representa "nada"

Hide Delegate:

evita cadenas largas con un método con las llamadas

Descompose Conditional:

Divide condicional grandes en métodos mas chicos

Duplicate Code

Definición: mismo código repetido en varios lugares

Refactoring:

- Extract Method
- Pull up Method
- Form Template Method

Large Class

Definición: una clase hace demasiado trabajo

Refactoring:

- Extract Class
- Extract SubClass

Long Method

Definición: método con demasiadas líneas

Refactoring:

- Extract Method
- Decompose Conditional
- Replace Temp with Query

Feature Envy

Definición: un método usa más datos de otra clase que de la propia

Refactoring:

- Move Method

Swithc Statements

Definición: Condicionales que controlan lógica según tipo de objeto

Refactoring:

- Replace Conditional with Polymorphism

Data Class

Definición: Clase que solo guarda datos sin lógica

Refactoring:

- Encapsulate Field
- Self Encapsulate Field
- Move behavior to Class

Long parameter list

Definición: métodos con demasiados parámetros

Refactoring:

- Introduce Parameter Object
- Preserve Whole Object

Magic Numbers

Definición: número misteriosos en el código

Refactoring:

- Replace Magic Number with Symbolic Constant

Middle Man

Definición: clase que solo delega llamadas sin aportar valor

Refactoring:

- Inline Method
- Remove Middle Man

Hierarchy Problems

Definición: jerarquias mal diseñadas, duplicación, clases innecesarias

Refactoring:

- Pull up Method/Field
- Push down Method
- Extract SuperClass/Subclass

Long Message Chain

Definición: llamadas encadenas a.getB().get().get()...

Refactoring:

- Hide Delegate
- Introduce Method

Lazy Class

Definición: clase que no hace casi nada y no justifica su existencia

Refactoring:

- Inline Class
- Collapse Hierarchy